MASTER PDF EDITION

Lemonade.gg Starter Guide

A beginner-to-advanced guide to what Lemonade.gg is, how it works, how to prompt it well, and how to use it effectively inside Roblox Studio.

Version 1.0.2

Source wiki 1385e3d

Single-file master export

Contents

This PDF is generated from the live GitHub wiki and arranged as one continuous handbook for reading, sharing, and archiving.

GUIDE OVERVIEW	Lemonade.gg: The Complete Dummies Guide
CHAPTER 1	Chapter 1: What Is Lemonade.gg?
CHAPTER 2	Chapter 2: Roblox Basics You Need to Know
CHAPTER 3	Chapter 3: Getting Started
CHAPTER 4	Chapter 4: What Lemonade Can Build
CHAPTER 5	Chapter 5: The Day-to-Day Workflow
CHAPTER 6	Chapter 6: Writing Great Prompts
CHAPTER 7	Chapter 7: Debugging and Fixing Issues
CHAPTER 8	Chapter 8: Building Your First Game
CHAPTER 9	Chapter 9: Tips, Tricks, and Advanced Usage
CHAPTER 10	Chapter 10: Limitations and What to Expect
CHAPTER 11	Chapter 11: Frequently Asked Questions
REFERENCE	Glossary

GUIDE OVERVIEW

Lemonade.gg: The Complete Dummies Guide

Welcome to the official Dummies Guide for **Lemonade.gg**, the AI coding tool built specifically for Roblox game developers. Whether you have never touched a line of code in your life or you have been building games on Roblox for years and just want to move faster, this wiki is for you.

This guide will walk you through everything from the very basics of what Lemonade is, all the way to advanced tips for getting the best results out of it. You can read it straight through like a book, or jump around to whatever section is most useful to you right now.

What Is This Guide?

This is a plain-English, beginner-friendly breakdown of how Lemonade.gg works, what it can do, and how to actually use it to build Roblox games. No fluff, no corporate speak, just real information written in a way that makes sense.

If you have been wanting to make a Roblox game but kept hitting a wall every time you had to write code, Lemonade is probably the tool you have been looking for.

A Note Before You Start

Lemonade.gg has a free tier and a paid Pro plan. The free plan gives you one credit per day for basic use. The Pro plan ($19.99/month) gives you faster generation, daily stacking credits, and larger project support. There are also one-time credit packs available if you want a credit top-up without committing to a subscription.

This guide is written based on what the tool does as of early 2026. If something in this guide does not match what you are seeing on the website, check the latest pricing and feature details at lemonade.gg. The core workflow stays consistent regardless of which plan you are on.

Who Should Read This Guide?

Honestly, anyone trying to build on Roblox who wants help with the coding side. But specifically this guide was written with these people in mind:

- **Total beginners** who have never written code and want to make a game
- **Builders and designers** who are great at making things look good but struggle with scripting
- **Experienced developers** who want to prototype and test ideas faster
- **Parents or teachers** helping younger creators get started
- **Content creators** who want to add custom features to their Roblox worlds

If you fit into any of those groups, you are in the right place.

Let's get into it. Start with Chapter 1 if you are brand new, or jump to whatever chapter fits where you are right now.

CHAPTER 1

Chapter 1: What Is Lemonade.gg?

The Short Version

Lemonade.gg is an AI-powered coding tool built specifically for Roblox game development. You describe what you want to add to your game in plain English, and Lemonade writes the Luau code for you and drops it directly into your Roblox Studio project. No copy-pasting, no Googling for tutorials, no spending three hours trying to figure out why your script broke.

That is really it at the core. Type what you want, get working code in your game.

But there is a lot more to it than that, and understanding what makes Lemonade different from other AI tools will help you get the most out of it. So let's dig in.

The Longer Version

Why Does This Tool Exist?

Roblox is one of the most popular game creation platforms in the world. Millions of people use it, and the barrier to entry for building something visually is actually pretty low. Roblox Studio gives you a lot of tools to place objects, design worlds, and set up the look and feel of your game without touching code at all.

The problem shows up the moment you want your game to actually *do* something.

Want a door that opens when a player walks up to it? You need a script. Want a leaderboard that tracks player points? Script. Want a shop where players can spend currency? Script. Want enemies that chase the player? Multiple scripts. Want those enemies to only chase the player if they are within a certain range, but back off if the player has a specific powerup active? You are writing a pretty complicated script now, and if you do not know Luau (Roblox's programming language), you are probably opening YouTube and losing an afternoon.

Lemonade was built to fix exactly this. It bridges the gap between having a creative vision for a game and being able to actually build it. The premise is simple: you should be able to focus on *designing* the game, and let the AI handle the code.

What Makes It Different From ChatGPT or Other AI Tools?

This is probably the first question most people have, and it is a fair one. If you have used ChatGPT or any general AI assistant, you know you can already ask it to write Roblox scripts. So what does Lemonade add?

Three things: context, accuracy, and automation.

Context is the big one. When you ask a general AI like ChatGPT to write a Roblox script, it has to work from general knowledge about Roblox's API. It does pretty well in a lot of cases, but it can make mistakes. It might use deprecated functions, get confused about which services handle what, or write code that technically looks right but does not actually work in Roblox. Lemonade uses proprietary Roblox-specific training data injected into every prompt, which makes it significantly more accurate for Roblox's actual API. It knows Roblox the way a specialist knows their field, rather than the way a generalist knows a little about everything.

Accuracy follows from that. Because Lemonade is purpose-built for Roblox and only Roblox, it understands the specific structure of a Roblox game. It knows what a Humanoid is. It knows the difference between a LocalScript and a Script. It knows where DataStore logic should live. It knows how RemoteEvents connect the client and the server. You do not have to explain any of this from scratch in your prompts.

Automation is the other major thing. When you use ChatGPT to write Roblox code, you get a block of text. You have to copy it, go to Roblox Studio, find the right place to put it, create a new Script or LocalScript, paste the code in, and then test it. That whole process takes time and also requires you to know where the code should go, which is not always obvious if you are new.

Lemonade skips all of that. It has a plugin that connects directly to Roblox Studio, and when Lemonade generates code, it pushes it to the correct location in your project automatically. You describe what you want, and a few seconds later it is in your game and ready to test.

Who Built It?

Lemonade.gg launched in late 2025 as the first dedicated AI coding tool for Roblox. It quickly became the number one ranked application in the Programming category on OpenRouter, which is a platform that tracks AI application usage. In its first 30 days of serious adoption, it processed over 382 billion tokens, which tells you there are a lot of developers using it actively.

It is not affiliated with Roblox Corporation. It is a third-party tool that integrates with Roblox Studio through their plugin system, similar to how tools like Rojo or other development utilities work.

Is It Legitimate? Is It Safe?

Yes on both counts.

Lemonade generates standard Luau code, the same programming language you would write by hand. Roblox does not prohibit the use of AI-assisted coding tools. The code is yours once it is generated, and it works exactly like code you would write yourself.

The plugin connects to Roblox Studio through Roblox's official plugin API, which means it operates within Roblox's own sandbox and permissions system. It does not have access to your account credentials, payment information, or anything outside of the Studio project you are working on.

From a trust perspective, it has strong adoption in the Roblox development community and a solid reputation. You can find creators and developers talking about it on YouTube, TikTok, and the Roblox developer forums.

How Much Does It Cost?

Lemonade has a free tier and a paid Pro plan.

Free Plan ($0/forever): 1 credit per day. Basic code generation. Good for casual users or people just getting started.

Pro Plan ($19.99/month): 40 credits upfront when you subscribe, plus a daily login bonus that stacks up to 5 credits per day. Faster generation speed, support for larger and more complex projects, up to 10 projects, and the PRO Discord role for priority support.

Credit Packs (one-time purchases): If you do not want a subscription but need more credits for a specific project, you can buy credit packs. $4.99 gets you 21 credits, $19.99 gets you 86 credits, and $99.99 gets you 438 credits. Note that credit packs include a 15% platform fee and do not unlock Pro plan features.

Pricing is handled through the Lemonade website and does not use Robux.

The Elevator Pitch

Here is how you would explain Lemonade to someone who knows nothing about Roblox development or AI tools:

Imagine you want to build a Roblox game. You have ideas. You know what it should look like and how it should feel to play. But every time you want to add a feature, you hit a wall because making things actually work requires writing code, and writing code takes time to learn and is easy to get wrong.

Lemonade lets you describe the feature in plain English, and it builds the code for you and puts it directly in your game. It understands Roblox specifically, not just coding in general. And because it lives inside Roblox Studio through a plugin, there is no fumbling around trying to figure out where to paste things. You describe, it builds, you test.

That is Lemonade.

Recap: Key Facts About Lemonade.gg

- It is an AI tool built specifically for Roblox game development
- It generates Luau code from plain English descriptions
- It syncs directly into Roblox Studio via a plugin
- It is more accurate for Roblox than general AI tools because it uses Roblox-specific training data
- Free tier available; Pro plan at $19.99/month with more credits and features
- It is fully compliant with Roblox's platform terms
- It is not made by Roblox, but it works with Roblox Studio
- It works for beginners and experienced developers alike

Ready to keep going? Chapter 2 covers the Roblox concepts you need to know before you start building.

CHAPTER 2

Chapter 2: Roblox Basics You Need to Know

Why This Chapter Exists

You do not need to be a Roblox developer to use Lemonade, but knowing a few basic concepts will make a huge difference in how well your prompts turn out. This chapter is not a full Roblox scripting tutorial. It is just the vocabulary and mental model you need to write good prompts and understand what Lemonade is doing when it generates code for you.

If you are already a Roblox developer with some experience, you can probably skim this or skip it entirely. If you are new, read through it at least once. It will save you a lot of confusion later.

The Big Picture: How a Roblox Game Is Structured

A Roblox game is not just a 3D world sitting on a server somewhere. It is a collection of objects, services, and scripts that all work together. Roblox calls this structure the **data model**, and it is organized in a hierarchy kind of like folders on your computer.

At the top of that hierarchy is the **game** itself, which contains a bunch of services. Each service handles a specific part of how the game works. Think of services like departments in a company: StarterGui handles user interface, Workspace holds all the 3D stuff in the world, ServerScriptService runs server-side logic, and so on.

Scripts live inside these services, and they tell the game what to do. Understanding where a script lives matters a lot because Roblox treats scripts differently depending on their location.

The Two Sides of a Roblox Game: Client and Server

This is probably the most important concept to understand, and it is one that trips up a lot of beginners.

Every Roblox game runs on two sides simultaneously: the **server** and the **client**.

The server is a computer run by Roblox that runs the actual game. It handles things that need to be consistent for everyone: the game world, player data, enemy behavior, anything that all players need to agree on. There is only one server per game session (for most games), and it is authoritative. What the

server says goes.

The client is the player's own computer. Each player connected to your game has their own client running. The client handles things that only that player needs to see: their own GUI, their camera, local animations, that kind of thing.

Why does this matter? Because a script running on the server behaves differently than a script running on a client. If you put a GUI on the server, players might not see it the way you expect. If you try to modify another player's character from the client, it won't work. Roblox separates these two sides for security and performance reasons.

When you write prompts for Lemonade, being specific about which side something should run on helps it generate better, more correct code.

The Main Services You Need to Know

Here are the services you will reference most often in your prompts:

Workspace

This is where the 3D world lives. Every part, model, and physical object in your game that you can see and interact with in the 3D view lives in Workspace. If you want code that interacts with objects in the world (like detecting when a player touches a part, or spawning enemies), it usually involves Workspace.

StarterPlayerScripts

Scripts placed here are copied to each player's client when they join the game. These run locally on each player's machine. Good for personal UI behavior, camera effects, or anything that should only affect that specific player.

StarterCharacterScripts

Similar to StarterPlayerScripts, but these scripts are re-inserted into the player's character every time their character spawns or respawns. Good for scripts that affect the player's character directly, like custom movement.

StarterGui

This is where you put your user interface. Anything you put here gets copied to each player's PlayerGui when they join. Buttons, menus, shop screens, health bars, and anything else on the screen belongs here.

ServerScriptService

Scripts here run on the server and are never visible to clients. This is the most secure place to put important server-side logic. Anything involving player data, game state, economy, or anything you do not want players to tamper with should go here.

ReplicatedStorage

This is a shared folder that both the server and clients can access. It is commonly used to store RemoteEvents, RemoteFunctions, and modules that need to be accessible from both sides.

ServerStorage

Like ReplicatedStorage, but only the server can access it. Good for storing things like enemy models or items that you do not want clients to be able to see directly.

Scripts, LocalScripts, and ModuleScripts

There are three types of scripts in Roblox:

Script

A regular Script runs on the server. It has access to server-side services and can affect the game for all players. If you put a Script in ServerScriptService, it runs on the server. If you put a Script inside a part in Workspace, it also runs on the server.

LocalScript

A LocalScript runs on a specific player's client. It can only directly affect things on that player's machine. It is how you make interactive GUIs, handle input, and build anything that is unique to one player's experience.

ModuleScript

A ModuleScript is different from the other two. It does not run on its own. Instead, it is a chunk of reusable code that other scripts can call on. Think of it like a library. If you have a function you want to use in multiple places, you put it in a ModuleScript and require it from other scripts. It is an advanced concept, but Lemonade uses them frequently when generating more complex systems.

RemoteEvents and RemoteFunctions

Because the server and client are separated, they need a way to talk to each other. That is what RemoteEvents and RemoteFunctions are for.

RemoteEvent: Used to send a message from the client to the server, or from the server to a client (or all clients). It is one-way communication. The recipient listens for the event, and the sender fires it. Example: a player clicks a button on their screen (client), which fires a RemoteEvent telling the server to give them coins (server handles the economy).

RemoteFunction: Similar, but it is two-way. The caller sends a request and waits for a response. Less common than RemoteEvents, but useful when you need to ask the server for information and get an answer back.

Both of these typically live in ReplicatedStorage so both the server and clients can find them.

When you ask Lemonade to build features that involve both the player's screen and game-wide effects, it will almost always use RemoteEvents. You do not need to know how to write them yourself, but knowing they exist helps you understand the code it generates.

DataStores

A DataStore is how you save player data between sessions. Without a DataStore, everything a player earns, unlocks, or achieves in your game is wiped the moment they leave.

DataStores are managed by Roblox's servers and are only accessible from server-side Scripts. They store data in key-value pairs, kind of like a simple spreadsheet. You give a player's data a key (usually their UserId), and you can save and load values associated with that key.

DataStore code can be tricky to write correctly because it involves asynchronous operations (things that take a little time to complete, like loading data from a remote server). Lemonade handles DataStore code well, which is one of its standout capabilities since this is an area where many beginners struggle.

Humanoid

The Humanoid is a special object that lives inside a player's character model. It controls health, movement, and a bunch of other character-related things. Almost every script that interacts with a player's body references the Humanoid at some point.

Common things the Humanoid controls:

- Health (Humanoid.Health)
- Walk speed (Humanoid.WalkSpeed)
- Jump power (Humanoid.JumpPower)
- Whether the character can jump or climb
- Animations playing on the character

If you ask Lemonade to add a speed boost, slow a player down, reduce health, or trigger a specific animation, it will use the Humanoid to do it.

The Explorer and Properties Panels

In Roblox Studio, the **Explorer** panel on the right shows you the entire hierarchy of your game, including all the services and everything inside them. When Lemonade pushes code into your project, you will see new Script objects appear in the Explorer. This is how you know it worked.

The **Properties** panel shows you the settings of whatever object you have selected. When you click on a script in the Explorer, you can see its name and other settings here.

You do not need to know these inside and out, but when Lemonade says "I placed a LocalScript in StarterPlayerScripts," you should know where to look to find it.

Luau: Roblox's Programming Language

Luau is the programming language Roblox uses. It is a version of Lua (a lightweight scripting language) that has been modified and extended by Roblox for performance and safety. It is simpler than most programming languages and reads fairly clearly even if you are not a developer.

Here is an example of a basic Luau script that prints a message when a player joins:

```
game.Players.PlayerAdded:Connect(function(player)
    print("Welcome to the game, " .. player.Name)
end)
```

You do not need to learn Luau to use Lemonade. But being able to look at generated code and understand roughly what it is doing will help you catch issues and write better prompts over time.

Core Terms to Remember

Here is the short list worth keeping in your head as you start using Lemonade. If you want the full reference definitions, jump to the Glossary.

Term	What It Is
Server	The Roblox-hosted computer that runs the game for all players
Client	The individual player's computer
Workspace	The 3D world container
StarterGui	Where UI elements go
ServerScriptService	Server-side scripts, hidden from clients
ReplicatedStorage	Shared storage accessible by both server and clients
RemoteEvent	Lets the server and client communicate
DataStore	Saves player data between sessions
Humanoid	Controls a player character's properties

You Do Not Need to Memorize All of This

Seriously. This chapter is here as a reference, not a test. As you start using Lemonade and building things, these terms will start clicking into place naturally. The main takeaways for now are:

1. There is a server and a client, and they are different
2. Scripts go in specific places and those places matter
3. GUIs go in StarterGui
4. Server logic goes in ServerScriptService

5. Saving player data uses DataStores

Hold onto those five things, and you will be in good shape for the chapters ahead.

Next up: Chapter 3 will walk you through signing up, installing the plugin, and making your first connection between Lemonade and Roblox Studio.

CHAPTER 3

Chapter 3: Getting Started

What You Need Before You Begin

Before you can use Lemonade, you need two things:

1. **Roblox Studio** installed on your computer
2. A **Lemonade.gg account**

That is it. You do not need to buy anything extra, install additional software, or already know how to code. Both are free to get started, and the setup process takes about ten minutes from start to finish.

Think of this as the **first-time setup** chapter. Once you are installed and connected, Chapter 5 covers the repeatable session workflow you will use every time you sit down to build.

Step 1: Getting Roblox Studio

If you do not already have Roblox Studio installed, here is how to get it:

1. Go to the Roblox website and log in to your Roblox account (or create one if you do not have one)
2. Click **Create** in the top navigation
3. Follow the prompts to download and install Roblox Studio

Roblox Studio is free for all Roblox users. Once it is installed, you can open it from your desktop or start menu like any other program.

If you already have Roblox Studio installed, make sure it is up to date. Roblox updates Studio regularly, and using an outdated version can sometimes cause plugin issues.

Step 2: Creating a Lemonade Account

1. Open your browser and go to **lemonade.gg**
2. Click **Sign in with Roblox**
3. You will be taken to the official Roblox website to authorize the connection
4. Approve the permissions and you will be redirected back to Lemonade

There is no separate Lemonade username or password. Your Roblox account is your Lemonade account. Lemonade only reads your display name and avatar from Roblox. Both permissions it requests are marked low risk on Roblox's side.

Sign-up is open to everyone with a Roblox account.

Step 3: Choosing Your Plan

When you sign up, you will be on the free plan by default. Here is what each plan gives you:

Free Plan ($0/forever): 1 credit per day. Good for getting started and testing things out.

Pro Plan ($19.99/month): 40 credits upfront plus a daily login bonus that stacks up to 5 credits per day. Faster generation, support for larger projects, up to 10 projects, and the PRO role on the Lemonade Discord for priority support.

Credit Packs: One-time credit top-ups if you want more without a subscription. $4.99 for 21 credits, $19.99 for 86 credits, $99.99 for 438 credits. Credit packs include a 15% platform fee and do not unlock Pro features.

You can start with the free plan and upgrade later. For a serious project, the Pro plan is worth it for the daily stacking credits alone.

Step 4: Installing the Lemonade Plugin in Roblox Studio

This is the step that makes everything work. The Lemonade plugin is what connects your Roblox Studio project to the Lemonade website. Without it, the code Lemonade generates would have no way to get into your game.

Here is how to install it:

1. Open **Roblox Studio**
2. Open any project (you can create a new blank baseplate if you do not have one yet)
3. In the top toolbar, find and click the **Toolbox** button
4. In the Toolbox panel, switch to the **Plugins** tab
5. Search for **Lemonade AI**
6. Click the plugin and then click **Install**

Once installed, you will see the Lemonade plugin appear in your Studio toolbar. It may show up as a button or a panel depending on the version.

A Note on Plugin Permissions

When you install any plugin in Roblox Studio, Studio will ask you to confirm you trust the plugin. Lemonade operates through Roblox's official plugin API, so it is subject to the same permission system as all other Studio plugins. It needs permissions to read and modify scripts in your project, which is how it syncs your generated code into the right places.

Step 5: Connecting Lemonade to Your Project

There is no login step inside the plugin. The connection works like this:

1. On **lemonade.gg**, create a project and make sure it is open
2. In **Roblox Studio**, click the Lemonade plugin button to open the plugin panel
3. Click **Connect**
4. Wait for the Status indicator to turn green and show as connected

Once connected, the plugin panel shows your project name, prompt count, and the last web sync time. The Lemonade website will then allow you to send prompts. Generated code syncs automatically into the correct places in your open Studio project in real time.

If you close and reopen Studio, just click Connect again with your project open on the website.

Step 6: Open Lemonade.gg in Your Browser

Lemonade's main interface is web-based. You type your prompts, see your generated code, and manage your project through the website at **lemonade.gg**.

Keep both Roblox Studio and the Lemonade website open at the same time. This is the basic dual-window setup you will use every time you work with Lemonade:

- **Roblox Studio** on one side (or on one monitor): your game project where scripts appear
- **Lemonade.gg** in your browser: where you type your prompts

You do not have to switch back and forth constantly. You can write your prompt on the website, Lemonade generates and syncs the code, and you can click Play in Studio to test without ever leaving Studio.

Your First Test

If you are brand new and just want to confirm everything is working, try this prompt:

"Add a script that prints 'Hello from Lemonade!' in the output window when the game starts."

This is one of the simplest possible scripts. Open the Output window in Roblox Studio (View > Output) and press Play. You should see the message appear. If it does, your connection is working and you are ready to go.

Setting Up a New Game vs. Working on an Existing One

Lemonade works with both.

Starting fresh: Create a new Baseplate or empty place in Roblox Studio. Open it, connect the plugin, and start prompting. Lemonade will generate all the scripts your game needs as you build.

Working on an existing game: Open your existing place file or game in Studio, connect the plugin, and start prompting. Lemonade can add new scripts to your existing project or even help you improve and debug scripts that are already there.

If you are working on an existing game that already has a lot of scripts, it helps to give Lemonade some context in your prompts. Tell it what already exists and what you want to add or change. The more specific you are, the better the results.

Troubleshooting the Setup

The plugin does not appear after installation Close and reopen Roblox Studio completely. Sometimes a fresh restart is needed for plugins to load.

The plugin shows up but will not connect Make sure you have a project open on lemonade.gg before clicking Connect in the plugin. If it still will not connect, check your internet connection and try clicking Connect again.

Code is not appearing in Studio after generating it Make sure the project you have open in Studio is the same one Lemonade is trying to sync to. If you have multiple projects open, close the others. Also check the Explorer panel in Studio to confirm whether the scripts are appearing there.

I see a permissions error in Studio This can happen if the plugin does not have the right permissions. Try reinstalling the plugin, and when prompted to confirm permissions, make sure you approve everything it asks for.

The Interface Overview

Once you are logged in and connected, the Lemonade web interface has a few key parts worth knowing:

The prompt input: This is where you type what you want to build. It works like a chat box. You can type in plain English, describe your feature, and hit send.

The conversation history: Lemonade keeps a running history of your prompts and its responses. This is useful for reviewing what you have asked for, seeing what code was generated, and understanding how your project has evolved.

Version history: Lemonade tracks previous versions of your project. If something breaks after a change, you can roll back to an earlier version.

You do not need to memorize every part of the interface right now. Chapter 5 revisits the pieces you will actually use during a normal work session.

One More Thing Before You Start Prompting

The most important thing to understand as a new user is that Lemonade works best when you give it one feature at a time. Do not open it up on your first day and type "build me a complete RPG game." Start with one thing, get comfortable with the workflow, and build up from there.

There is a whole chapter on writing great prompts in Chapter 6. Read that before you go too deep. The quality of what Lemonade produces is directly tied to the quality of what you ask for.

You are set up and ready. Chapter 4 covers everything Lemonade can actually build so you know what to ask for.

CHAPTER 4

Chapter 4: What Lemonade Can Build

The Short Answer

Lemonade is primarily a tool for two things: **UI (user interfaces)** and **code (scripting)**. Those two categories cover a huge percentage of what goes into making a functional Roblox game, which is why the tool is so useful in practice.

Within those two categories, it can handle an impressive range of features. This chapter breaks down what Lemonade is good at, gives you examples of real things you can ask it to build, and helps you understand what kinds of projects it is best suited for.

User Interfaces (GUI)

This is one of Lemonade's strongest areas. Roblox GUI design requires a combination of layout skills and scripting knowledge, and getting both right manually is one of the more time-consuming parts of building a game. Lemonade handles the full picture: the visual layout, the button logic, the transitions, and the underlying scripts that make everything interactive.

What kinds of GUIs can it make?

Shop screens: Full purchase UIs where players can spend in-game currency on items, upgrades, or boosts. This includes the layout, the item listings, the buy buttons, and the logic that checks if the player has enough currency and applies the purchase.

Leaderboards: On-screen player rankings showing stats like kills, points, coins, level, or any custom value you track. Lemonade can build both the visual display and the data hooks that keep it updated in real time.

Health bars and stat displays: HUD elements that show the player's current health, mana, stamina, XP, or any other tracked stat. These update live as the values change in game.

Main menus: Title screens, lobby screens, and any UI a player sees before they are spawned into gameplay. Can include play buttons, settings, credits, and more.

Inventory screens: Panels that show items the player is carrying, equipped, or has collected. This can range from simple list-based displays to more complex grid-based inventory layouts.

Settings menus: Controls for things like sound volume, graphics quality, and keybinds. These can write their values back to DataStores so the settings persist between sessions.

Quest and objective panels: Displays that track active quests, objectives, progress counters, and completion states.

Notification systems: Toast popups, banners, and small alerts that appear temporarily to notify the player of events (earned a reward, leveled up, etc.).

Countdown timers: In-game timers displayed on screen for game rounds, ability cooldowns, or events.

The depth of what Lemonade can produce for GUIs is genuinely impressive. It understands things like AutomaticSize, UIListLayout, UICorner, UIStroke, ScrollingFrame behavior, and how to anchor elements properly. You can ask it for very specific visual styles, and it will translate them into Luau.

A Real Example: The Quest Panel

One tested example from **Silverfox0338** (the author of this guide) was an Infinity Castle-themed quest panel built around a dark fantasy / ancient scroll aesthetic. The prompt did not just ask for "a cool quest UI." It specified the font, the full parchment-and-ink color palette, text strokes, text size constraints, section dividers, accent bars, collapsible sections, scrolling behavior, and the exact left-side HUD positioning.

That matters because it shows what Lemonade is actually good at in practice. It can take a tightly defined visual direction and translate it into real Roblox UI structure: AutomaticSize, UIListLayout stacking, ScrollingFrame behavior, and interactive sections that feel tailored to a specific game rather than like a generic placeholder menu.

The result was a hand-crafted themed panel that worked on the first attempt, which is a strong example of how far Lemonade can go when you give it a clear target. If you want the full tested prompt and a step-by-step breakdown of why it works, Chapter 6 includes the complete example.

Scripting and Game Mechanics

Beyond UI, Lemonade handles a wide range of gameplay scripting. Here are the categories it covers well:

Player Systems

Movement modifiers: Speed boosts, slow effects, sprint mechanics, custom jump behavior. These interact with the Humanoid directly.

Character customization: Scripts that apply accessories, change body colors, resize characters, or alter appearance based on game state.

Health and damage systems: Scripts that deal damage, heal players, add invincibility frames, or create custom death behavior.

Respawn logic: Custom spawn points, spawn selection screens, or delayed respawns after death.

Economy and Data

In-game currency: Systems that give players coins, gems, or any other currency and track how much they have.

DataStore integration: Save and load player data so progress persists between sessions. Lemonade writes the full DataStore boilerplate including error handling for failed reads/writes.

Shop backend logic: The server-side logic behind purchases: checking if the player has enough currency, applying the item, deducting the cost.

Leaderboard data hooks: Server scripts that track and update player stats displayed on leaderboards.

World and Environment

Proximity prompts: Buttons that appear when players walk near something (like an interactable NPC, a door, or a collectible).

Touch events: Scripts that trigger when a player steps on or touches a specific part. Useful for damage zones, teleporters, checkpoints, and collectibles.

Teleportation: Scripts that move a player from one location to another, either within the same place or to a different game.

Day/night cycles: Changing the Lighting settings over time to simulate a day/night cycle.

Doors and gates: Parts that open, close, or move based on player input or game state.

Combat

Melee attacks: Hitbox detection, damage application, and cooldowns for melee combat systems.

Ranged attacks: Projectile creation, travel, and hit detection.

Ability systems: Cooldown-based special abilities that players can activate.

Enemy AI basics: Simple enemy behaviors like patrolling, chasing a player when they get close, and backing off or attacking when in range.

Progression

Level systems: XP gain, level-up logic, and stat scaling that improves as the player levels up.

Unlock systems: Features, items, or abilities that become available once the player hits a certain milestone.

Achievement tracking: Systems that monitor player actions and trigger rewards when conditions are met.

Wave and Round Systems

Wave spawners: Scripts that spawn groups of enemies in waves, increasing difficulty over time.

Round timers: Countdown-based round structures with start, end, and intermission phases.

Game state management: Scripts that handle when a game is waiting for players, running, ending, and resetting.

What Lemonade Is Especially Good At

Based on how developers use it, a few areas stand out as places where Lemonade consistently performs well:

DataStore code: This is one of the harder things to write by hand because it involves async operations and edge cases. Lemonade handles this reliably and writes defensive code that accounts for common failure modes.

GUI layouts: Even complex, multi-element UI panels with scrolling, collapsible sections, and dynamic content come out well. The ability to specify a visual style in detail and have it implemented accurately is a real strength.

Shop systems: The combination of UI and backend logic required for a shop is something Lemonade does end-to-end. You ask for a shop, and you get the screen, the buttons, the currency check, the item application, and the DataStore save all in one generation.

RemoteEvent wiring: Features that require communication between the client and server need properly structured RemoteEvents. Getting this right by hand requires understanding the client/server split clearly. Lemonade wires this up correctly without you needing to think about it.

A Quick Reference List

Category	Examples
GUIs	Shop, leaderboard, inventory, HUD, main menu, quest log, settings
Player	Speed boost, damage, heal, sprint, custom spawn, respawn delay
Economy	Currency system, shop backend, DataStore saves, purchase logic
World	Doors, teleporters, touch events, proximity prompts, day/night
Combat	Melee hitbox, projectile, ability cooldown, basic enemy AI
Progression	XP system, level-up, unlocks, achievements
Rounds/Waves	Wave spawner, round timer, game state management

---

Next: Chapter 5 walks you through the actual step-by-step process of using Lemonade on a real project.

CHAPTER 5

Chapter 5: The Day-to-Day Workflow

How a Lemonade Session Actually Works

Understanding the full workflow from start to finish before you begin saves a lot of confusion. This chapter walks you through what a real Lemonade session looks like, from opening your tools to testing a finished feature in Studio.

The good news is that the workflow is simple and repeatable once you get used to it. Most developers fall into a rhythm pretty quickly.

Chapter 3 covered the one-time setup. This chapter is about the repeatable rhythm you use after that initial setup is already done.

Session Prep (Do This Each Time You Work)

This is the quick pre-flight check before a work session, not the full onboarding process from Chapter 3.

1. Open Roblox Studio Launch Studio and open your game project. If you are starting fresh, create a new Baseplate or empty place.

2. Open Lemonade.gg in Your Browser and Open Your Project Navigate to lemonade.gg and open the project you want to work on. The project needs to be open on the website before you connect from Studio.

3. Connect the Plugin Click the Lemonade button in the Studio toolbar to open the plugin panel, then click Connect. Wait for the Status indicator to turn green. Once it does, the plugin is live and any code Lemonade generates will sync into your open Studio project automatically.

4. Arrange Your Windows The ideal setup is having Roblox Studio and your browser side by side, or on separate monitors if you have them. You want to be able to type a prompt, send it, and immediately see the result in Studio.

The Interface at a Glance

Once you are connected, these are the parts of the interface you will actually touch during a normal session:

The left sidebar has three icon buttons:

- A monitor icon showing your plugin connection status (green = connected)
- A clock/history icon for your chat history
- A cursor/square icon to open the UI Editor

The Inventory sits to the left of the chat input. This is a deck of pre-built component cards that you can use as starting points. Each card represents a ready-made mechanic like "Kill Brick," "Camera Effects," or "Ball Physics." Clicking a card flips it to show what it does, and you can use it as the base for a prompt.

The chat input is the main text box where you type your prompts. The placeholder text says "Describe a game mechanic..." The bottom of the input has buttons for:

- Tagging a file from your project with @
- Uploading an image
- Opening the UI Editor
- Selecting your AI model
- Viewing your credit balance
- Sending the prompt

The model selector lets you choose which AI model Lemonade uses to generate your code. There are four options: Step 3.5 Flash (free), Gemini 3 Flash (low cost, the default), Claude Sonnet 4.5 (moderate cost), and Claude Opus 4.6 (highest cost). For most everyday tasks, Gemini or Sonnet are the right choices. Opus is the most powerful but is expensive and should only be used for unusually complex code that other models cannot handle. See Chapter 9 for a full breakdown.

The Core Loop

Once you are set up, the workflow for adding each feature is:

1. Decide what you want to build Take thirty seconds to think clearly about what the feature needs to do, who it affects (the player, the server, everyone?), and what it should connect to in the game.

2. Write your prompt In the chat input, type a clear description of what you want. Be specific. Include details about where the feature should appear, what it should do, and any conditions that matter. More on this in Chapter 6.

Alternatively, pick a card from the Inventory as a starting point if there is one that matches what you are building.

3. Send the prompt and wait Lemonade processes your prompt and generates the appropriate Luau code. This takes anywhere from a few seconds to about fifteen seconds depending on complexity.

4. Watch Studio Check the Explorer panel in Roblox Studio. You should see new Script or LocalScript objects appearing in the right locations as the plugin syncs the generated code directly into your project.

5. Click Play and test Press the Play button in Studio to run the game. Test the feature you just added. Does it work as expected? Are there any errors in the Output window?

6. Iterate if needed If something is not quite right, go back to Lemonade and describe the problem or the adjustment. Lemonade will update the relevant scripts.

7. Move on to the next feature Once the feature works the way you want, pick the next thing to build and repeat.

The Inventory Cards

The Inventory card deck is one of Lemonade's more unique features. Each card is a pre-built, ready-to-deploy mechanic. Cards have a front showing the name and a preview image, and a back with a short description of what they do.

Examples of cards you might see in the Inventory:

- **Kill Brick**: make a kill brick
- **Ball Physics**: kick the ball by walking into it
- **Camera Effects**: set up dark lighting and fog

You can click on a card to flip it and read the description, then use it as the foundation for a prompt. This is useful when you know the general shape of what you want but need a starting point, or when you want to quickly add a common mechanic without writing a prompt from scratch.

The Inventory grows over time as more components are added.

The UI Editor

The UI Editor is a separate mode accessible from the sidebar icon or from the button inside the chat input. It gives you a visual workspace for designing GUIs before generating code for them.

This is useful when you have a specific visual layout in mind but find it easier to sketch it out than to describe it in text. You can use the UI Editor to define the structure and approximate look of your interface, and then let Lemonade generate the actual Luau code to implement it.

Choosing Your AI Model

The model selector in the chat input lets you pick which AI model powers your generations. The default is Gemini 3 Flash, and for most everyday work it is a solid choice.

If your prompt is more detailed, visual, or complex, Claude Sonnet is usually the better step up. For a full model-by-model breakdown, see Chapter 9.

A Concrete Example: Adding a Coin System

Here is what the full workflow looks like for building a basic coin collection system.

What we want to build:

- Coins scattered around the map that players can collect by touching them
- Each coin gives the player 10 coins
- The coin total is tracked and displayed on screen
- The coin total saves when the player leaves

Prompt 1: The coin pickup

> "Create a Script in ServerScriptService that detects when a player touches a part tagged 'Coin' in Workspace, gives them 10 coins using a leaderstats IntValue, and destroys the part after collection."

Lemonade generates the script. It appears in ServerScriptService in the Explorer. You test it by pressing Play, walking your character into a part named Coin, and checking the leaderstat in the top corner of the screen.

Prompt 2: The DataStore save

> "Add a DataStore to the coin system so each player's coin total saves when they leave and loads when they join. The DataStore should handle failures gracefully."

Lemonade updates the existing script (or generates a new one) to include the DataStore logic.

Prompt 3: The on-screen display

> "Create a GUI in StarterGui that shows the player's current coin count in the bottom left corner of the screen. It should update in real time as the player collects coins."

Lemonade generates a LocalScript and a ScreenGui. The coin counter appears on screen when you press Play.

Three prompts. A working coin collection system with persistence and a live display. That is the workflow.

Version History: Your Safety Net

Every prompt and its corresponding code generation is saved. If you ask Lemonade to change something and it breaks a feature that was working before, you can roll back to the previous version.

Use version history as a safety net when:

- A new generation breaks something that worked
- You want to experiment with a different approach without losing the current one
- You need to review what exactly changed after a particular prompt

---

Fixing Errors

When something does not work after testing, check the Output window in Roblox Studio (View > Output). Copy the error message and paste it into the Lemonade chat with a note like "Fix this error:" followed by the error text. Lemonade will identify the problem and correct the script.

Tips for a Smooth Workflow

Test after every prompt: Do not generate ten things in a row and then try to test them all at once. You will not know which one broke something.

Save your Studio project frequently: Lemonade syncs code into your place, but you still need to save the place file itself (Ctrl+S) to make sure your project is stored safely.

Use Play mode for quick tests: For most single-player logic, the standard Play button is fine and faster. Use Team Test when you need to test multiplayer behavior or client/server communication.

Keep sessions focused: Pick one feature to build per session if possible. Jumping between multiple unfinished features leads to a messy project.

Next: Chapter 6 covers how to write prompts that consistently get you great results.

CHAPTER 6

Chapter 6: Writing Great Prompts

Why Prompts Matter So Much

Lemonade is only as good as what you ask it for. The AI behind it is powerful and Roblox-aware, but it cannot read your mind. If you give it a vague description, you will get something that is technically functional but probably not what you had in mind. If you give it a specific, detailed description, you will get exactly what you were imagining.

This chapter is about developing the skill of writing prompts that consistently produce great results. It is probably the single most important skill to build when using Lemonade.

The Golden Rule: One Feature at a Time

Before anything else, internalize this: **ask for one feature per prompt**.

The most common mistake new users make is trying to build the whole game in a single prompt. They type something like "make me a full RPG with a shop, combat system, inventory, quests, leveling, and boss fights" and then wonder why the result is incomplete or disorganized.

Lemonade can build all of those things. Just not all at once in one go. Break your game into discrete features and build them one by one. This approach:

- Gives you clean, isolated code for each feature
- Makes testing easier because you know what you just changed
- Makes debugging easier because problems are easier to locate
- Lets you roll back a single feature without losing everything else

Build incrementally. Every professional game developer works this way regardless of whether they use AI tools.

The Anatomy of a Good Prompt

A strong prompt for Lemonade has three components:

1. What you want to build

Be explicit about the feature. Not "make a GUI" but "make a shop GUI with three item slots." Not "add a damage script" but "add a script that deals 15 damage to a player when they step on a part tagged 'LavaTile'."

2. Where it should go

Specify the location in the Roblox hierarchy. "In StarterGui." "In ServerScriptService." "A LocalScript in StarterPlayerScripts." Lemonade knows where things should generally go, but being explicit removes ambiguity and ensures the code lands exactly where you expect.

3. What it connects to or depends on

If the feature needs to interact with something that already exists in your game, say so. "Connect it to the 'PlayerData' leaderstats." "The button should fire the 'PurchaseItem' RemoteEvent that already exists in ReplicatedStorage." "It should read from the player's Coins IntValue."

Going from Vague to Specific

Here are examples of vague prompts transformed into specific ones:

Vague: Make a shop. **Specific**: Create a shop GUI in StarterGui with three items: a Speed Boost for 50 coins, a Double Jump for 75 coins, and a Shield for 100 coins. Each item should have a buy button. When clicked, it should check if the player has enough coins using their Coins leaderstats value, apply the item effect, and deduct the cost.

Vague: Add enemies. **Specific**: Create a server Script in ServerScriptService that spawns an NPC enemy every 30 seconds at a part named 'EnemySpawn' in Workspace. The enemy should walk toward the nearest player when they are within 40 studs and deal 10 damage every 2 seconds if they get within 5 studs. Destroy the enemy when its health reaches 0.

Vague: Make a save system. **Specific**: Add a DataStore to ServerScriptService that saves each player's Coins, Level, and XP IntValues from their leaderstats when they leave the game, and loads them back when they join. Handle DataStore errors gracefully with a pcall.

Vague: Add a timer. **Specific**: Create a LocalScript in StarterPlayerScripts that displays a countdown timer in a TextLabel at the top center of the screen. The timer starts at 120 seconds and counts down to 0. When it hits 0, fire a RemoteEvent named 'RoundEnd' to the server.

Specifying Location in the Hierarchy

These phrases help Lemonade put code in the right place:

- "A Script in ServerScriptService"
- "A LocalScript in StarterPlayerScripts"
- "A LocalScript in StarterGui, inside a ScreenGui named 'ShopGui'"
- "Add this to the existing script in ServerScriptService named 'GameManager'"
- "Inside the model named 'EnemyBase' in Workspace"

If you do not specify a location, Lemonade will make a reasonable guess. But specifying always gives you more control.

Writing Prompts for Visual Styles

For GUI work specifically, you can be extremely detailed about appearance. Lemonade handles color, font, size, layout, and visual effects from written descriptions.

Here is what a highly detailed visual style prompt looks like. This prompt was written by **Silverfox0338** (the author of this guide) for a dark fantasy / Infinity Castle-themed game UI, and has been tested and confirmed to produce working output:

```
Create a Roblox GUI in Luau using the following visual style, this is a dark
fantasy / ancient scroll aesthetic used in an Infinity Castle-themed game UI.

FONT: Use Enum.Font.Fondamento exclusively throughout the entire UI (titles,
descriptions, objectives, buttons, labels, icons). This is non-negotiable, do not
use GothamBold, Arial, or any other font.

COLOR PALETTE (Color3.fromRGB):
- Titles / Headers:       (215, 190, 150)  warm parchment gold
- Descriptions:           (170, 165, 175)  muted dusty lavender
- Objective text:         (195, 190, 200)  soft silver-violet
- Completed items:        (145, 195, 155)  sage green
- Progress counters:      (155, 185, 210)  pale steel blue
- Fold/icon accents:      (180, 160, 130)  aged bronze
- Divider lines:          (160, 130,  95)  dark tawny brown
- Accent bars/borders:    (175, 145, 105)  warm ochre leather
- Text outline stroke:    (  5,   3,   8)  near-black ink

TEXT STROKES: Every TextLabel and TextButton must have a UIStroke child with
Color = Color3.fromRGB(5, 3, 8) and Thickness = 1.5. This gives the carved-ink
look essential to the style.

TEXT SIZE CONSTRAINTS: Use UITextSizeConstraint on all labels:
- Titles: MinTextSize=12, MaxTextSize=20, TextSize=18
- Body/Objectives: MinTextSize=11, MaxTextSize=16, TextSize=15

LAYOUT:
- BackgroundTransparency = 1 on all frames (no solid backgrounds)
- Use UIListLayout for vertical stacking with Padding = UDim.new(0.012, 0)
- Use AutomaticSize = Enum.AutomaticSize.Y on all frames that contain text
- ClipsDescendants = true on collapsible body sections

DIVIDERS: Between sections use a Frame (Height=2px, Width=90%, centered) with:
- BackgroundColor3 = Color3.fromRGB(160, 130, 95)
- BackgroundTransparency = 0.15
- A UIGradient with Transparency fading to 1 at both ends (0.9 at 0%, 0 at 10%, 0
```

```
at 90%, 0.9 at 100%)
- A thin glow frame (Height=1px) above it with the same gradient at lower opacity

ACCENT BARS: Left-side vertical accent lines on section headers use a 3px-wide
Frame with:
- BackgroundColor3 = Color3.fromRGB(175, 145, 105)
- BackgroundTransparency = 0.2
- UICorner with CornerRadius = UDim.new(0, 2)

ICONS / SYMBOLS: Use Unicode glyphs as TextLabels with the Fondamento font:
- Collapse arrows:  v (open) / > (closed)
- Incomplete items: diamond outline
- Complete items:   filled diamond (with color changed to sage green)

SCROLLING: Use ScrollingFrame with:
- ScrollBarThickness = 2
- ScrollBarImageColor3 = Color3.fromRGB(140, 115, 85)
- ScrollBarImageTransparency = 0.4
- CanvasSize = UDim2.fromScale(0, 0)
- AutomaticCanvasSize = Enum.AutomaticSize.Y

COLLAPSIBLE SECTIONS: Each quest/section should have a fold button that toggles a
Body frame's Visible property. Animate the arrow glyph between the two states on
click.

POSITIONING: Anchor the main container at (0, 0) with Position =
UDim2.fromScale(0.012, 0.12) and Size = UDim2.fromScale(0.22, 0.72) for a left-
side HUD panel layout.

The overall feel should be: aged manuscript, candlelit, hand-inscribed, like
quest notes written on parchment and pinned to a dungeon wall. No modern UI
elements, no rounded card containers, no drop shadows, no solid background
panels.
```

This prompt is long, but look at what it accomplishes: it defines every single visual decision up front. The result of a prompt this specific is a GUI that matches the game's aesthetic precisely rather than something generic that needs to be tweaked a dozen times afterward.

A note on model choice and cost: a detailed prompt like this, run on Claude Sonnet 4.5 through Lemonade's model selector, costs less than a single credit and produces a result that would take hours to build by hand. For detailed GUI work like this, Sonnet is the recommended model.

You do not need to write prompts this detailed for every feature. But for something as visually important as a core HUD element or a signature piece of UI, putting in this level of detail saves a huge amount of back-and-forth later.

The "What, Where, When, Who" Framework

When you are stuck on how to phrase a prompt, run through these four questions:

What does it do? Describe the behavior or appearance of the feature.

Where does it live? Which service, which container, which object?

When does it happen? On player join? On touch? On button click? On a timer?

Who does it affect? The individual player, all players, a specific team, the server?

If you can answer all four, you have everything you need for a solid prompt.

Using Images in Prompts

Lemonade supports image uploads in the chat input. This is useful when you want to:

- Show a reference screenshot of a UI layout you want to replicate
- Describe a visual style by showing an example rather than describing it in words
- Share a screenshot of an error or unexpected behavior

Click the image icon in the chat input to attach an image to your prompt.

Tagging Files

The @ button in the chat input lets you tag specific files from your project. This is useful when you want Lemonade to modify or reference a specific existing script. Instead of just naming it in text, you can tag it directly so Lemonade has the exact file in context.

Adding a Validation Clause

A useful trick for complex features is to add a validation clause at the end of your prompt:

> "Make sure all functionality from the previous prompts still works correctly after this change."

Or:

> "The existing DataStore save logic should remain intact."

This signals to Lemonade that you are aware of what is already in the project and do not want it to break. It acts as a self-check in the generation.

Referencing Existing Scripts

If you want Lemonade to modify or extend something it already generated, reference it directly:

> "Edit the Script in ServerScriptService named 'CoinSystem' to also give the player 1 XP each time they collect a coin."

> "Add a second item to the shop GUI we created earlier: a Jetpack for 200 coins."

> "The DataStore save system we built earlier is not saving the player's Level. Fix that."

What NOT to Do

Do not be vague about numbers: "Give the player some coins" is worse than "Give the player 25 coins."

Do not stack multiple features in one prompt: This produces disorganized code that is harder to test and debug.

Do not use ambiguous names: "The button" means nothing without context. "The Buy button inside the ShopFrame ScreenGui" is specific.

Do not forget to mention what already exists: If a feature needs to interact with something in your game, tell Lemonade it is there.

Prompting for Fixes and Adjustments

Something looks wrong visually:

> "The coin counter GUI is appearing in the center of the screen instead of the bottom left. Move it to the bottom left corner with a small margin from the edge."

Something does not work:

> "The proximity prompt on the door is appearing but pressing it doesn't open the door. Fix the door open script."

An error appeared in the output:

> "Fix this error that appears when a player joins: [paste the error text here]"

The Prompt Checklist

Before you send a prompt, run through this quick check:

- ☐ Is this one feature, not several at once?
- ☐ Did I specify what it does?
- ☐ Did I specify where it should go in the hierarchy?
- ☐ Did I specify when it triggers?
- ☐ Did I mention anything it needs to connect to?
- ☐ Are my numbers and values specific?
- ☐ If modifying something existing, did I name it?

Ready to look at what happens when things go wrong? Chapter 7 covers how to diagnose and fix errors.

CHAPTER 7

Chapter 7: Debugging and Fixing Issues

AI-Generated Code Needs Testing

AI-generated code is a starting point, not a finished product. Simple features often work perfectly on the first try. More complex systems, or systems that depend on other things already in your project, may need a round or two of fixes before they are working exactly right.

This is normal. Even experienced developers who write code by hand need to debug and test. The difference with Lemonade is that it can help you debug too, which speeds the whole process up considerably.

The Output Window: Your First Debugging Tool

The Roblox Studio Output window is the most important tool for figuring out what went wrong. If you do not have it open, go to **View > Output** in Studio to open it.

When you press Play and something does not work, the Output window will usually show you an error message like this:

```
ServerScriptService.CoinSystem:14: attempt to index nil value 'Player'
```

Breaking this down:

- **ServerScriptService.CoinSystem** tells you which script has the problem
- **:14** tells you it is on line 14
- **attempt to index nil value 'Player'** tells you what went wrong

You can see these errors in Lemonade's built-in console monitor too, so you do not always have to flip back to Studio to check.

How to Use Lemonade to Fix Errors

The fastest way to fix a Lemonade-generated script is to feed the error directly back to Lemonade:

> "I'm getting this error when testing: [paste the full error message here]. Fix it."

Or with more context:

> "The coin pickup script is throwing this error when a player touches a coin: [error message]. The script should be giving the player 10 coins and destroying the coin part."

Lemonade will identify the cause and update the script. This loop of test, find error, feed error back, get fix is a completely normal part of the workflow.

Common Errors and What They Mean

"attempt to index nil value"

Something in the script is trying to access a property on something that does not exist. The most common error in Roblox scripting.

Common cause: The script is looking for an object before it exists, or with the wrong name.

Fix prompt: "Fix this nil value error on line [X]: [error]. The object it is looking for is [describe what it should be finding]."

"attempt to call a nil value"

The script is trying to call a function that does not exist. Often happens when a function name is misspelled or a module is not set up correctly.

Fix prompt: "Fix this nil function error: [error]. The function it should be calling is [describe the intended behavior]."

"DataModel or Instance is read-only"

The script is trying to modify something it does not have permission to modify. Often happens when client-side code tries to change something only the server should control.

Fix prompt: "Fix this read-only error: [error]. The script should be modifying [object] from the server side, not the client."

"infinite yield possible"

A warning that a script is waiting for something that might never appear. Can cause your game to hang.

Fix prompt: "I'm getting an 'infinite yield' warning for [object name]. Make sure the script properly handles the case where this object might not be available immediately."

"Stack overflow"

A script is calling itself in an infinite loop.

Fix prompt: "The script is causing a stack overflow. Here is the code that's causing it: [paste the relevant section]. Fix the infinite loop."

GUI Not Appearing

Not always accompanied by an error. Common reasons:

- The ScreenGui was created in the wrong location (e.g., in ReplicatedStorage instead of StarterGui)
- BackgroundTransparency is 1 on a frame that should be visible
- The frame size or position puts it off-screen
- A script is supposed to make it visible but is not running

Fix prompt: "The shop GUI is not appearing when I press Play. The ScreenGui is in StarterGui. Check why it might not be showing and fix it."

DataStore Not Saving

Common causes:

- DataStores only work in published games or live servers, not in Studio Play mode by default (you need to enable Studio Access to API Services)
- The PlayerRemoving event fires before the data finishes saving if the save is async and not awaited properly

Fix prompt: "The player's coin data is not saving between sessions. The DataStore save runs on PlayerRemoving. Make sure the save is awaited properly so it completes before the player fully disconnects."

Enabling Studio DataStore Access

If your DataStore scripts are not working in Studio testing, enable API access:

1. In Roblox Studio, go to **File > Game Settings**
2. Click the **Security** tab
3. Enable **Allow HTTP Requests** and **Enable Studio Access to API Services**

Without this, DataStore calls will silently fail in Studio.

Debugging Without an Error Message

Sometimes things just do not work and there is no error. Check these:

Check the Explorer panel: Are the scripts actually in the project? Sometimes a sync issue means the code did not land where expected.

Check script names: If your script references an object by name, make sure the object in Studio has exactly that name, including capitalization.

Check which mode you are testing in: Some features only work in Team Test mode, especially anything involving client/server communication.

Add print statements: Ask Lemonade to add `print()` statements at key points in the script so you can trace execution in the Output window.

> "Add print statements throughout the shop purchase logic so I can trace exactly when each step runs in the output."

Check RemoteEvent names: If a client fires an event and the server does not respond, the RemoteEvent names might not match exactly between the firing script and the listening script.

When to Roll Back vs. When to Fix Forward

Roll back when: The new generation fundamentally broke something that was working, and the bug is not obvious. Trying to patch a broken foundation usually creates more problems.

Fix forward when: There is a clear, specific error that Lemonade can address in a targeted way.

The threshold: if you can write a one-sentence description of the exact problem, fix forward. If you need a paragraph to describe all the ways it went wrong, roll back.

Debugging Complex Systems

When a system has multiple parts (a GUI, a RemoteEvent, a server script, a DataStore save), isolate one part at a time. Ask Lemonade to confirm each piece works individually before you connect them together. This is slower but saves you from chasing an error that turns out to be in an unexpected place.

Next: Chapter 8 puts everything together into a guided walkthrough of building a simple but complete Roblox game from scratch.

CHAPTER 8

Chapter 8: Building Your First Game

What We Are Building

This chapter is a complete guided walkthrough of building a simple but functional Roblox game using Lemonade. By the end, you will have a working game with:

- A coin collection system
- A leaderboard showing each player's coin count
- A shop where players can spend coins on a speed boost
- Player data that saves between sessions
- A basic enemy that chases players

This covers the core systems that appear in almost every Roblox game: economy, UI, combat/enemies, and persistence. Once you can build these, you can build almost anything by expanding on them.

The prompts here are written in full so you can copy them directly if you want to follow along.

Before You Start

Open Roblox Studio, create a new Baseplate place, connect the Lemonade plugin, and open lemonade.gg in your browser. Make sure the plugin connection indicator is green.

In Studio, add a few Parts to the Workspace and name them "Coin" (right-click in the Explorer > Rename). Give them any color or shape you like. These are the collectible coins players will pick up.

Add another Part somewhere in the world and name it "EnemySpawn". This is where enemies will spawn from.

Step 1: Leaderstats

Before anything else, set up a leaderstats folder. Leaderstats is a Roblox convention where a folder named "leaderstats" inside a player displays stats in the default leaderboard overlay in the top right corner.

Prompt 1:

> Create a Script in ServerScriptService that sets up leaderstats for every player when they join. Each player should have three IntValues in their leaderstats folder: Coins (starting value 0), Level (starting value 1), and XP (starting value 0). Name the script 'LeaderstatSetup'.

Check the Explorer. You should see 'LeaderstatSetup' in ServerScriptService. Press Play and you should see the leaderboard overlay appear with each player's stats.

Step 2: Coin Collection

Prompt 2:

> Create a Script in ServerScriptService that detects when a player's character touches any part named 'Coin' in Workspace. When a player touches a Coin, give them 10 Coins using the Coins IntValue in their leaderstats. Then destroy the Coin part. Use a debounce to prevent the same player from collecting the same coin twice. Name the script 'CoinPickup'.

Press Play, walk your character into a Coin part, and watch the leaderboard. The Coins value should go up by 10 and the part should disappear.

If it does not work, check the Output window for errors. The most common issue is if the Coin parts are not named exactly "Coin" (Roblox names are case-sensitive).

Step 3: DataStore for Persistence

Right now, if a player leaves and rejoins, their coins reset to zero. Fix that with a DataStore.

Prompt 3:

> Add a DataStore to ServerScriptService that saves each player's Coins, Level, and XP IntValues from their leaderstats when they leave the game. When they rejoin, load the saved values and apply them to their leaderstats. Use a DataStore named 'PlayerData'. Handle DataStore errors with pcall. Use the player's UserId as the key. Name this script 'DataManager'.

To test DataStores in Studio, enable Studio Access to API Services (File > Game Settings > Security > Enable Studio Access to API Services).

Test by earning coins in Play mode, stopping Play, pressing Play again, and confirming the coins carried over.

Step 4: The Shop GUI

Prompt 4:

> Create a shop GUI in StarterGui. The shop should be a ScreenGui named 'ShopGui'. Add a button in the top right area of the screen labeled 'Shop' that opens and closes the shop panel when clicked. The shop panel should contain one item: 'Speed Boost' for 50 Coins. The buy button should be labeled 'Buy - 50 Coins'. Use a dark-themed color scheme with rounded corners on the panel and buttons. Name the main script 'ShopClient' and place it as a LocalScript inside StarterGui.

After this generates, press Play and you should see a Shop button in the top right corner. Clicking it should open a panel with the Speed Boost option.

Step 5: The Shop Backend

The GUI exists but buying does not do anything yet. Add the server-side logic.

Prompt 5:

> Create a server Script in ServerScriptService named 'ShopServer'. When a player attempts to purchase the Speed Boost, check if they have at least 50 Coins in their leaderstats. If yes, deduct 50 Coins and apply a speed boost by setting their Humanoid's WalkSpeed to 24 for 30 seconds, then reset it to the default 16. Use a RemoteEvent named 'PurchaseItem' in ReplicatedStorage to receive the purchase request from the client.

Prompt 6 (connecting client to server):

> Update the ShopClient LocalScript in StarterGui so that when the player clicks the Buy button for the Speed Boost, it fires a RemoteEvent named 'PurchaseItem' in ReplicatedStorage with the item name 'SpeedBoost'. Create the RemoteEvent in ReplicatedStorage if it does not already exist.

Now press Play, collect 50 or more coins, open the shop, and click Buy. Your character should move faster for 30 seconds.

Step 6: A Basic Enemy

Prompt 7:

> Create a Script in ServerScriptService named 'EnemySpawner'. Every 30 seconds, spawn a simple NPC enemy at the Part named 'EnemySpawn' in Workspace. The NPC should be a basic rig with a Humanoid. It should walk toward the nearest player when they are within 40 studs. If the NPC gets within 5 studs of a player, deal 5 damage to that player every 1.5 seconds. The NPC should have 50 health. When the NPC's health reaches 0, destroy it. Give the NPC a red color so it is visually distinct.

Enemy AI can sometimes need a round of fixes. If the enemy does not move, check the Output for errors and feed them back to Lemonade.

Step 7: A Win Condition

Prompt 8:

> Update the CoinPickup script in ServerScriptService so that when a player's Coins value reaches 500, fire a RemoteEvent named 'PlayerWon' from the server to that specific client.

Prompt 9:

> Create a LocalScript in StarterPlayerScripts named 'WinHandler' that listens for the 'PlayerWon' RemoteEvent from ReplicatedStorage. When it fires, create a brief full-screen notification with large text saying 'YOU WIN!' that fades out after 5 seconds.

Testing the Full Game

Press Play and test the complete loop:

1. Walk around collecting Coin parts and watch the leaderboard go up
2. Open the shop and buy a Speed Boost once you have 50 coins
3. Watch the enemy spawn after 30 seconds and chase you
4. Collect coins until you hit 500 and see the win screen
5. Leave and rejoin to confirm your coin count was saved

If everything works, you have built a complete functional Roblox game using nothing but plain English prompts.

What to Build Next

From here, obvious expansions include:

More shop items: Ask Lemonade to add more items using the same pattern.

Better UI: Use the prompt style from Chapter 6 to give your shop and leaderboard a custom visual design that fits your game's theme.

Enemy drops: Make enemies drop a coin when they die.

Multiple rounds: Wrap the whole thing in a round structure with a countdown and reset.

Each of these is another handful of prompts. The game grows one feature at a time.

Chapter 9 has additional strategies and advanced techniques for getting even more out of Lemonade.

CHAPTER 9

Chapter 9: Tips, Tricks, and Advanced Usage

Tip 1: Name Everything Consistently

Lemonade generates code that references objects by name. If your naming is inconsistent, scripts will break because they are looking for "ShopButton" when the actual object is named "shopbutton" or "Shop Button."

Pick a naming convention early and stick to it:

PascalCase: ShopFrame, CoinPickup, EnemySpawner, MainMenuGui **camelCase**: shopFrame, coinPickup, enemySpawner, mainMenuGui

Either works. Consistency is what matters.

Tip 2: Start With Structure, Then Fill In Behavior

When building a complex system, prompt for the structure first and the behavior second.

Example: Building an inventory system

Prompt 1 (structure): "Create a ScreenGui named 'InventoryGui' in StarterGui with a main panel that has 20 item slots arranged in a 4x5 grid using a UIGridLayout. Each slot should be a 64x64 square Frame with a placeholder gray background. Add an open/close button for the panel."

Prompt 2 (behavior): "Now add a LocalScript inside InventoryGui that populates the grid slots with items from a player inventory table. Each slot should show the item name as a TextLabel and highlight when clicked."

Prompt 3 (backend): "Add a server Script in ServerScriptService that manages player inventory data using a DataStore. When a player picks up an item part in the world, add it to their inventory data and fire a RemoteEvent named 'InventoryUpdated' to that player's client."

Separating structure from behavior from backend gives you cleaner code and makes each piece easier to test independently.

Tip 3: Use ModuleScripts for Shared Data

If you have data that multiple scripts need to access (like a list of shop items with names and prices), put it in a ModuleScript in ReplicatedStorage.

> "Create a ModuleScript in ReplicatedStorage named 'ItemData' that returns a table containing all shop items. Each item should have a Name, Price (in Coins), and Description field. Include: Speed Boost (50 coins), Double Jump (75 coins), Shield (100 coins), Jetpack (200 coins)."

Now any script on the server or client can `require` this module. When you want to add or change items, you change them in one place.

Tip 4: Ask for Code Comments When Learning

By default, Lemonade generates clean code without excessive comments. If you are learning and want to understand what the code does, ask for it:

> "Generate this script with inline comments explaining what each major block does."

Reading commented code that you prompted yourself is one of the fastest ways to actually learn Luau.

Tip 5: Build a Template Project

Once you have a set of core systems that you reuse across games (leaderstats, DataStore save/load, basic RemoteEvent setup, a simple shop), save that as a template place file. For your next game, open the template and start from a foundation that already works.

Tip 6: Use "Refactor" Prompts to Clean Things Up

As a project grows, you can ask Lemonade to reorganize:

> "The CoinSystem script in ServerScriptService has grown to 200 lines and handles both coin pickup and the DataStore save. Split it into two separate scripts: 'CoinPickup' for touch detection and giving coins, and 'DataManager' for all DataStore operations."

> "Move all the GUI animation code from the shop script into a ModuleScript named 'ShopAnimations' in ReplicatedStorage."

Tip 7: Ask for Edge Case Handling

When a script first generates, it handles the happy path. Ask Lemonade to strengthen it:

> "Update the shop purchase script to handle the case where a player tries to buy an item they already have. Show a message saying 'You already have this item.'"

> "Update the enemy spawner to handle the case where no players are in the game when the enemy tries to find a target."

> "Add handling to the DataStore save so that if the save fails on the first attempt, it retries up to 3 times before logging a warning."

Tip 8: Use Lemonade for Rapid Prototyping

One of Lemonade's best use cases is testing whether a game mechanic feels fun before investing time building it properly:

> "Create a quick test version of a grappling hook mechanic. When the player presses Q, they should shoot toward the nearest wall within 50 studs and be pulled to it. It does not need to be polished, just functional enough to test if it feels good."

If it feels fun, refine the prompt and ask for a proper version. If it does not feel right, you spent two minutes finding that out instead of two hours.

Tip 9: Chain Prompts With Context

When you are deep in a session, you can refer back to earlier work naturally:

> "Remember the enemy spawner we built earlier? Add a second enemy type to it. The new enemy should be blue, move faster (speed 20 instead of 14), but have only 25 health. Spawn one of each type alternately."

> "Update the coin counter GUI from earlier to also show the player's Level and XP in the same panel, below the coin count."

Lemonade maintains conversation context so you do not need to re-explain what already exists.

Tip 10: Choosing the Right Model

The model selector in the chat input lets you pick which AI powers your generation. There are four options, and choosing wisely saves credits and gets better results.

Step 3.5 Flash (Free): Costs zero credits. Good for very simple, quick tasks where you just want to test something fast. Not the strongest for complex scripts or detailed GUIs.

Gemini 3 Flash (low cost): The default model. Fast and capable. Handles the majority of everyday scripting tasks and basic GUIs well. A solid choice for most of your prompts.

Claude Sonnet 4.5 (moderate cost): The recommended step up for anything more involved. Significantly better at following detailed visual specifications, complex multi-part systems, and GUI work where precision matters. Despite costing more than Gemini, it is still very credit-efficient for the quality it delivers. If a Gemini result is not quite right, try Sonnet.

Claude Opus 4.6 (highest cost): The most powerful model available, but also the most expensive by a wide margin. **Do not use Opus for everyday tasks.** Reserve it only for genuinely complex, unusual code that the other models are struggling with. If Sonnet cannot produce what you need after a couple of attempts, that is when Opus is worth the cost. Using it for simple scripts or routine UI work is a waste of credits.

The practical rule: Use Gemini for quick everyday tasks. Use Sonnet for anything GUI-heavy or complex. Only reach for Opus when Sonnet genuinely cannot get the job done.

Tip 11: Use the Inventory Cards as Starting Points

The Inventory card deck in the interface has pre-built components for common mechanics. Instead of writing a prompt from scratch for something like a kill brick or camera effect, check if there is a card for it first.

Cards are a good starting point, but you can always follow up with a prompt to customize or extend what they produce.

Tip 12: Testing Multiplayer Behavior

The regular Play button only simulates a single player. To test multiplayer:

1. In Studio, go to **Test > Team Test**
2. This opens a local server and client as separate windows
3. Add more client windows from the Team Test controls

Test anything involving RemoteEvents, shared leaderboards, enemy behavior affecting multiple players, or shop purchases in Team Test mode.

Tip 13: Keep Chats Under 25 to 30 Prompts

The longer a conversation gets, the more context Lemonade has to process with each new prompt. Past around 25 to 30 prompts in a single chat, credit costs per generation start climbing noticeably because each prompt is carrying the weight of the entire conversation history.

If you are deep into a session and notice generations feeling heavier or costing more, start a fresh chat. You do not lose your project, just the conversation thread. Briefly recap what you have already built in the first prompt of the new chat to give Lemonade the context it needs.

Keeping chats focused and reasonably short also tends to produce cleaner results. Long conversations with lots of back-and-forth can introduce context noise that affects generation quality.

Advanced: Asking for Architecture Advice

Lemonade can help you think through how to structure a complex system before you start building:

> "I want to build a class system where players can choose from three classes: Warrior, Mage, and Rogue. Each class has different base stats and a unique ability. How should I structure this in Roblox? Give me the architecture before writing any code."

Lemonade will describe an approach: where to store the class data, how to apply stats, how to handle abilities, what the client/server communication should look like. Then follow up with "Build it using that structure."

This two-step approach leads to cleaner, more intentional code for complex systems.

Chapter 10 is honest about what Lemonade cannot do.

CHAPTER 10

Chapter 10: Limitations and What to Expect

Being Honest About What Lemonade Is

Lemonade is a genuinely useful tool, and it can dramatically speed up Roblox game development. But it is not magic. Understanding what it cannot do, and what it sometimes struggles with, will save you frustration and help you set realistic expectations.

What Lemonade Is Built For

To be direct about it: Lemonade is built for **UI and scripting (code)**. Those are its two core strengths, and that is where you should focus your expectations.

Practically everything on the scripting and interface side of a Roblox game falls within its capability. But there is a whole other side to game development that Lemonade does not touch at all.

What Lemonade Cannot Do

3D Modeling and Asset Creation

Lemonade cannot build your world. It cannot create 3D models, design terrain, or produce meshes. If your game needs a custom sword model, a dungeon environment, or a stylized NPC rig, you will need to build that in Roblox Studio's terrain tools, import external models, or use assets from the Roblox Creator Store.

Think of it this way: Lemonade builds the logic layer of your game. The visual/physical layer is still on you.

Animation

Creating new character animations, rigging a custom rig, or authoring animation tracks is entirely outside Lemonade's scope. You would use Roblox's Animation Editor for simple animations or export to an external tool like Blender for complex ones.

Audio

Lemonade cannot compose music or produce sound effects. You can ask it to write scripts that play specific sound IDs, but the sounds themselves need to be uploaded separately.

Anything Outside Roblox

Lemonade is purpose-built for Roblox only. It understands Luau, Roblox services, and Roblox's API. If you want to build a Unity game, a web app, or anything else, Lemonade is the wrong tool.

Limitations Within Its Scope

Even within what Lemonade does (UI and code), there are areas where it works better than others.

Very Complex Multiplayer Synchronization

For most games, Lemonade handles client/server communication correctly. But extremely complex systems that require very precise synchronization across many players simultaneously (competitive shooter hit detection, real-time physics simulations that must be identical for all players) can be tricky. These are hard to get right even for experienced developers, and AI-generated code for these scenarios may need significant manual refinement.

Cutting-Edge Roblox APIs

Roblox adds new services and APIs regularly. If you are trying to use something very recently added to Roblox, Lemonade may not have full knowledge of it yet. Check the Roblox developer documentation for the latest specifics on brand-new features.

Third-Party Scripting Frameworks

If your project depends heavily on a specific third-party framework (like Knit, Nevermore, or another scripting framework), Lemonade may generate vanilla Luau that does not integrate cleanly with the framework's conventions.

Generated Code Needs Testing

The code Lemonade generates is a starting point. It is not guaranteed to work perfectly on the first generation, especially for complex features.

Simple, isolated scripts often work on the first try. Multi-part systems that need to communicate with other things in your project may need a round or two of testing and adjustment.

The workflow is: generate, test, identify issues, prompt for fixes. It is faster than writing code by hand, but it is not a one-click "make game" button.

The Credit System

Lemonade uses a credit system. Here is what you need to know:

Free Plan ($0/forever)

- 1 credit per day
- Basic code generation
- Good for casual use or getting started

Pro Plan ($19.99/month)

- 40 credits upfront when you subscribe, plus a daily login bonus that stacks up to 5 credits per day
- Faster generation
- Supports larger, more complex projects
- Up to 10 projects
- PRO role on the Lemonade Discord for priority support

Credit Packs (one-time)

- $4.99 for 21 credits
- $19.99 for 86 credits (+1% bonus)
- $99.99 for 438 credits (+3% bonus)
- 15% platform fee is included in these prices
- Credit packs add credits only and do not unlock Pro features

A practical note on credit cost: simple prompts cost fewer credits. A basic script uses less than a detailed GUI with multiple sections. Even highly detailed prompts like the quest panel example from Chapter 6 are cost-efficient when run on a model like Claude Sonnet.

Managing Expectations for Beginners

You will still benefit from knowing some basics: Lemonade removes the need to write code, but understanding where scripts go and how the client/server model works will make your prompts better. Chapter 2 was written for exactly this reason.

Not every feature will be perfect immediately: Expect a few rounds of prompting for complex features.

Some things take multiple prompts: Large systems (full combat, full economy, complex AI) are built across several prompts, not just one.

The tool is not a replacement for game design thinking: Lemonade builds features you ask for. It does not tell you what your game should be, what mechanics will be fun, or how to balance difficulty. Those decisions are still yours.

A Realistic Summary

Lemonade is excellent at:

- Building GUIs from descriptions, including complex styled ones
- Writing game logic scripts quickly and correctly
- Handling DataStore boilerplate
- Debugging errors when given clear error messages
- Wiring up RemoteEvents between client and server

Lemonade is less reliable for:

- Very complex real-time multiplayer synchronization
- Integrating with third-party scripting frameworks
- Very large monolithic "build everything at once" requests

Lemonade cannot help with:

- 3D assets, models, terrain, or maps
- Animations
- Audio
- Anything outside the Roblox platform

Chapter 11 answers the most common questions people have before and after getting started.

CHAPTER 11

Chapter 11: Frequently Asked Questions

General Questions

Do I need to know how to code to use Lemonade?

No. Lemonade is genuinely beginner-friendly and is designed specifically for people who want to make Roblox games without coding experience. You describe what you want in plain English and it handles the code.

That said, knowing a little about how Roblox is structured will make your prompts better. Chapter 2 covers everything you need to know before you start.

Is Lemonade only for beginners?

Not at all. Plenty of experienced Roblox developers use it to speed up prototyping and handle repetitive scripting tasks. DataStore boilerplate, GUI layout generation, and wiring up RemoteEvent infrastructure are all things that are faster with Lemonade even if you could write them yourself.

Is Lemonade made by Roblox?

No. Lemonade is a third-party tool made independently of Roblox Corporation. It integrates with Roblox Studio through Roblox's official plugin API, the same system used by other third-party tools.

Is it safe to use for published games? Will Roblox ban me for using it?

Yes, it is safe. The scripts Lemonade generates are standard Luau code, no different from code you would write yourself. Roblox does not prohibit AI-assisted coding, and Lemonade's plugin operates within Roblox's own sandbox.

Does it work for any game genre?

Yes, as long as the feature you are building involves UI or Luau scripting. It works equally well for simulators, obby games, RPGs, tycoons, fighting games, horror games, and anything else. For the limits of what it cannot do, see Chapter 10.

Pricing Questions

What does Lemonade cost, in short?

Free gives you 1 credit per day. Pro costs $19.99/month and adds 40 upfront credits, a daily stacking bonus up to 5 credits per day, faster generation, larger project support, up to 10 projects, and the PRO Discord role. Credit packs are one-time top-ups if you want more credits without subscribing. Chapter 1 and Chapter 3 cover the full plan breakdown.

What counts as a credit?

Each prompt you send that generates code uses credits. Simpler prompts cost fewer credits. Complex, long-form generations cost more. Even highly detailed prompts are generally inexpensive per generation.

What payment methods are supported?

Lemonade uses real currency, not Robux. Debit/Credit Card and Cash App are supported directly through the Polar checkout. PayPal and crypto are not directly supported.

If your card is being declined, the most reliable workaround is usually a virtual VISA gift card. That is still easier and cheaper than trying to route payment through Robux.

How long do credits take to arrive after purchase?

Credits can take up to 45 minutes to appear after a confirmed payment. If it has been longer than that, open a support ticket.

Can my balance go negative?

Yes. Lemonade allows generations to continue even if your credit balance hits zero, so a task does not cut off partway through. Your balance will go negative and you will need to earn or purchase credits to bring it back up.

When should I buy Pro vs. a credit pack?

For regular use, the Pro plan's daily stacking bonus makes it the better value. Credit packs are better for occasional users who want a larger buffer for a specific project without a subscription.

Technical Questions

How does the Studio plugin work?

The Lemonade plugin creates a live bridge between Roblox Studio and the Lemonade website. When you generate code on the website, the plugin inserts the generated scripts into the correct locations in your open Studio project automatically.

Can I use Lemonade on an existing project, or does it need a fresh game?

You can use it on either. Open any existing Studio project, connect the plugin, and start prompting. Lemonade can add new features, fix bugs in existing scripts, or help you extend systems you already built.

It works best when you are explicit about what already exists. If you name the current script, object, or system you want changed, Lemonade has much better context.

What should I check if sync is not working?

Make sure the correct project is open on lemonade.gg, the plugin is connected in Studio, and the green monitor icon is showing as live. If sync still fails, reconnect the plugin. As a fallback, you can always copy the generated code from the conversation history and insert it manually.

What are the Inventory, UI Editor, and model selector?

- **Inventory**: A deck of pre-built mechanic cards you can use as prompt starting points or deploy directly.
- **UI Editor**: A visual workspace for sketching GUI layouts before generating the final code.
- **Model selector**: The choice of which AI model powers the generation. Gemini is the everyday default, Sonnet is the usual step up for more detailed work, and Opus is for unusual high-complexity cases.

Chapter 5 walks through the interface, and Chapter 9 covers model choice in more detail.

Usage Questions

Is there a prompt character limit?

Yes. The maximum prompt length is 20,000 characters. If your prompt is longer than that, break it into smaller focused messages. Shorter, targeted prompts tend to produce better results anyway.

Can I collaborate with someone on the same game?

You cannot share a project or credits with another user on Lemonade. However, you can invite someone as a collaborator in Roblox Studio directly, and both of you can use your own Lemonade accounts and credits on the same game simultaneously.

What are checkpoints and does restoring one refund credits?

The checkpoint button above the chat input reverts the changes made by your most recent prompt. It is a quick undo for when a generation breaks something. Restoring a checkpoint does not refund the credits spent on that prompt, since credits reflect the actual cost of the generation.

How do I get free credits beyond the daily one?

There are a few ways to earn extra free credits:

- **Step 3.5 Flash**: The free model costs zero credits. Use it for simple tasks to save your paid credits for harder ones.

- **QA Insiders**: Active community members selected to test upcoming features receive 10 to 20 credits per week.
- **Game jams**: Lemonade occasionally hosts game jams where winners receive credits or other prizes.
- **Giveaways**: After major updates, Lemonade sometimes runs credit giveaways.

Watch announcements for game jams and giveaways.

How specific do my prompts need to be?

The more specific, the better. Vague prompts get functional but generic code. Specific prompts with exact values, object names, and behavior descriptions get code that does exactly what you want.

The most common beginner mistake is trying to build the whole game in one prompt. Ask for one feature at a time. Chapter 6 covers this in detail.

Can I improve or replace code Lemonade already wrote?

Yes. You can ask it to refactor, optimize, add edge case handling, reorganize a script, or fix a specific bug. If the result is still not right, roll back using version history and ask for a more targeted revision.

How do I get support if something is not working?

The best place is the Lemonade Discord server. Pro plan users get the PRO Discord role which comes with priority support.

Almost done. The Glossary has quick definitions for all the terms used throughout this guide.

REFERENCE

Glossary

A quick reference for all the terms used throughout this guide. If you come across a word you do not recognize while reading, this is where to look it up.

A

API (Application Programming Interface) A defined set of functions and rules that lets software communicate with another system. When this guide mentions Roblox's API, it means the collection of services, functions, and properties that Luau scripts can use to interact with the Roblox engine.

AutomaticSize A property in Roblox GUI that causes a frame or other element to automatically resize to fit its contents. Commonly used in text-heavy UI so panels grow or shrink based on how much content is inside them.

B

Baseplate The default starting template in Roblox Studio. It is a flat gray platform with basic settings. A good starting point for new projects.

BackgroundTransparency A property of GUI elements that controls how see-through they are. A value of 0 is fully opaque (solid color), a value of 1 is fully transparent (invisible background). Lemonade-generated GUIs often use transparency to create layered visual effects.

C

Client The player's own computer running their copy of the game. Each player has their own client. LocalScripts run on the client. See also: Server.

ClipsDescendants A property on GUI frames that prevents child elements from rendering outside the frame's boundaries. Used for collapsible panels so content that slides in/out stays visually contained.

Credit The unit of usage in Lemonade. Each generation consumes credits. The Free plan gives 1 credit per day. The Pro plan gives 40 upfront plus a daily stacking bonus.

D

DataModel The complete hierarchy of a Roblox game, including all services, parts, scripts, and objects. When you see the Explorer panel in Studio, you are looking at the data model.

DataStore A Roblox service that lets you save and retrieve player data between game sessions. Uses key-value storage on Roblox's servers. Requires API access to be enabled in game settings for Studio testing.

Debounce A programming technique used to prevent a block of code from running multiple times in rapid succession. In Roblox, commonly used in touch detection to prevent a player from triggering the same event over and over by standing on a part.

E

Explorer The panel in Roblox Studio that shows the full hierarchy of your game. Everything in your place appears here as a tree of objects. When Lemonade syncs code, new scripts appear here.

G

GUI (Graphical User Interface) Everything the player sees on their screen that is not part of the 3D game world. Buttons, menus, leaderboards, health bars, and shop panels are all GUI elements.

I

Inventory (Lemonade) A card-based system in the Lemonade interface that contains pre-built mechanic components. Each card represents a ready-to-use feature like Kill Brick, Camera Effects, or Ball Physics. Cards can be flipped to read a description, and used as a starting point for a prompt or deployed directly into your project.

H

Humanoid A special Roblox object inside player (and NPC) character models that controls health, movement speed, jump power, and other character properties. Most scripts that interact with a player's body reference the Humanoid.

L

Leaderstats A convention in Roblox where a folder named "leaderstats" inside a player's instance displays that player's stats in the default scoreboard overlay in the top right corner of the screen. IntValues inside leaderstats appear as columns on the scoreboard.

LocalScript A type of Roblox script that runs on a specific player's client. Used for GUI behavior, input handling, and anything that should only affect one player's view of the game. Cannot directly modify objects that are server-controlled.

Luau The programming language used in Roblox. It is based on Lua with modifications and extensions by Roblox for performance and safety. All scripts in Roblox are written in Luau.

M

ModuleScript A type of script that does not run on its own. It contains reusable code (functions, tables, data) that other scripts can load using `require()`. Useful for shared data and utility functions used across multiple scripts.

N

NPC (Non-Player Character) A character in the game world controlled by scripts rather than by a player. Enemies, shopkeepers, and decorative characters are all NPCs. Lemonade can generate basic NPC AI behavior like patrolling and chasing players.

P

PathfindingService A Roblox service that calculates navigable paths through the game world, allowing NPCs to move around obstacles intelligently rather than walking directly through walls. Used for more advanced enemy AI.

Plugin A third-party extension that adds functionality to Roblox Studio. The Lemonade plugin is installed through Studio's Toolbox and establishes the sync connection between Studio and the Lemonade website.

Pro Plan Lemonade's paid subscription tier at $19.99/month. Includes 40 upfront credits, a daily login bonus that stacks to 5 credits per day, faster generation, larger project support, up to 10 projects, and the PRO Discord role.

Prompt The plain-English description you type into Lemonade describing what you want to build. The quality of a prompt directly affects the quality of what Lemonade generates.

Properties Panel The panel in Roblox Studio that shows the settings and attributes of whatever object you have selected. Used to inspect and manually edit object properties.

R

RemoteEvent A Roblox object used to send messages between the server and a client (or all clients). Fires one-way: either server-to-client or client-to-server. Typically stored in ReplicatedStorage. Essential for any feature that involves both a GUI action and a server-side response.

RemoteFunction Similar to RemoteEvent but two-way: the caller sends a message and waits for a response. Used when the client needs to request information from the server and get an answer back.

ReplicatedStorage A Roblox service whose contents are accessible by both the server and all clients. A common place to store RemoteEvents, RemoteFunctions, and shared ModuleScripts.

S

Script A standard Roblox script that runs on the server. Has access to server-side services and can affect the game state for all players. Contrasted with LocalScript (runs on a client) and ModuleScript (does not run on its own).

ScreenGui The top-level container for GUI elements in Roblox. A ScreenGui placed in StarterGui gets copied to each player's PlayerGui when they join. All 2D on-screen elements live inside a ScreenGui.

Server The Roblox-hosted computer that runs the game. There is one server per active game session (for standard games). Scripts in ServerScriptService run here. The server is authoritative and cannot be tampered with by clients.

ServerScriptService A Roblox service where server-side Scripts are placed. Scripts here are never sent to clients, making it the most secure place for important game logic like economy systems, DataStore management, and game state.

ServerStorage A Roblox service accessible only by the server. Good for storing things like enemy models or item objects that players should not be able to directly inspect.

StarterCharacterScripts Scripts placed here are automatically inserted into each player's character when they spawn or respawn. Used for scripts that need to run relative to the character model.

StarterGui A Roblox service that holds GUI elements. When a player joins, the contents of StarterGui are copied to their PlayerGui, making the UI visible on their screen.

StarterPlayerScripts Scripts placed here are copied into each player when they join. These LocalScripts run on the player's client for the duration of the session.

T

Toolbox A panel in Roblox Studio that allows you to search for and install assets, models, and plugins from the Roblox ecosystem. Used to install the Lemonade plugin.

U

UI Editor (Lemonade) A visual design mode in the Lemonade interface accessible from the sidebar or the chat input. Lets you sketch out GUI layouts before generating code for them. Useful when you have a specific visual structure in mind and find it easier to design visually than to describe in text.

UICorner A Roblox GUI modifier that adds rounded corners to frames and buttons. Commonly used in modern UI design. Applied as a child object of the frame you want rounded.

UIGradient A Roblox GUI modifier that applies a color or transparency gradient to a frame. Used in Lemonade-generated UIs for effects like fading dividers.

UIGridLayout A Roblox GUI modifier that arranges child elements in a grid pattern. Used for inventory slots, item grids, and anything that needs a consistent row/column layout.

UIListLayout A Roblox GUI modifier that stacks child elements vertically or horizontally in a list. The most common layout tool for menus, panels, and any list of items.

UIStroke A Roblox GUI modifier that adds an outline stroke around text or frames. Used extensively for stylized UI, like the carved-ink text effect described in the Chapter 6 prompt example.

UITextSizeConstraint A Roblox GUI modifier that clamps a TextLabel's font size between a minimum and maximum value. Prevents text from getting too small on small screens or too large on large ones.

W

Workspace The Roblox service that contains the 3D game world. Every physical object, part, model, and terrain element lives in Workspace. Scripts that interact with in-world objects (like enemies, coins, or doors) almost always reference Workspace.

Z

ZIndex A property of GUI elements that controls which element appears on top when two elements overlap. Higher ZIndex values appear in front of lower ones. Used to manage layering in complex UI layouts.

That is everything in the glossary. Head back to Home to navigate to any chapter.